

Complete OOP Practical Programs

Practical 1 (part A).pdf

Java Calculator Program - Code and Output

Program Code:

```
import java.util.InputMismatchException;
import java.util.Scanner;
public class Calculator {
    static double addition(double a, double b) {
        return a + b;
    }
    static double subtraction(double a, double b) {
        return a - b;
    }
    static double multiplication(double a, double b) {
        return a * b;
    }
    static double division(double a, double b) {
        if (b == 0) {
            throw new ArithmeticException("Division by zero is not allowed.");
        }
        return a / b;
    }
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int choice;
    boolean continueCalc = true;
    do {
        try {
            System.out.println("\n--- Calculator Menu ---");
            System.out.println("1. Addition");
            System.out.println("2. Subtraction");
            System.out.println("3. Multiplication");
            System.out.println("4. Division");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            if (choice == 5) {
                System.out.println("Exiting Calculator. Thank you!");
                break;
            }
            System.out.print("Enter first number: ");
            double num1 = sc.nextDouble();
            System.out.print("Enter second number: ");
            double num2 = sc.nextDouble();
            double result;
            switch (choice) {
                case 1:
                    result = addition(num1, num2);
                    System.out.println("Result = " + result);
                    break;
                case 2:
                    result = subtraction(num1, num2);
                    System.out.println("Result = " + result);
                    break;
                case 3:
                    result = multiplication(num1, num2);
                    System.out.println("Result = " + result);
                    break;
                case 4:
                    result = division(num1, num2);
                    System.out.println("Result = " + result);
                    break;
                default:
                    System.out.println("Invalid choice! Please select 1 to 5.");
            }
        } catch (InputMismatchException e) {
            System.out.println("Invalid input! Please enter numeric values only.");
            sc.nextLine();
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        }
    } while (continueCalc);
    sc.close();
}
```

```
    }  
}  
Sample Output:  
Sample Output:  
--- Calculator Menu ---  
1. Addition  
2. Subtraction  
3. Multiplication  
4. Division  
5. Exit  
Enter your choice: 1  
Enter first number: 10  
Enter second number: 5  
Result = 15.0  
--- Calculator Menu ---  
Enter your choice: 4  
Enter first number: 10  
Enter second number: 0  
Error: Division by zero is not allowed.  
--- Calculator Menu ---  
Enter your choice: 5  
Exiting Calculator. Thank you!
```

practical 1 (part B).pdf

Single Inheritance in Java (Code + Output)

Java Code:

```
class Animal {
    void eat() {
        System.out.println("Animal is eating.");
    }
    void sleep() {
        System.out.println("Animal is sleeping.");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking.");
    }
    void showActions() {
        System.out.println("--- Dog Actions ---");
        eat();
        sleep();
        bark();
    }
}
public class SingleInheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.showActions();
    }
}
```

Output:

Output:

```
--- Dog Actions ---
Animal is eating.
Animal is sleeping.
Dog is barking.
```

practical 2 (Part B).pdf

Interface Polymorphism Demo (Java)

Java Code:

```
interface Shape {
    void draw();
    double area();
}
class Circle implements Shape {
    double radius;
    Circle(double radius) {
        this.radius = radius;
    }
    public void draw() {
        System.out.println("Drawing a Circle");
    }
    public double area() {
        return Math.PI * radius * radius;
    }
}
class Rectangle implements Shape {
    double length, width;
    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    public void draw() {
        System.out.println("Drawing a Rectangle");
    }
    public double area() {
        return length * width;
    }
}
}
public class InterfacePolymorphismDemo {
    public static void main(String[] args) {
        Shape s;
        s = new Circle(4.5);
        s.draw();
        System.out.println("Circle Area: " + s.area());
        System.out.println();
        s = new Rectangle(5, 8);
        s.draw();
        System.out.println("Rectangle Area: " + s.area());
    }
}
```

Output:

Drawing a Circle

Circle Area: 63.61725123519331

Drawing a Rectangle

Rectangle Area: 40.0

practical 2 (part A).pdf

Java Ecommerce Program - Code and Output

Program Code:

```
import java.util.Scanner;
class Product {
    String name;
    double price;
    int quantity;
    Product() {
        name = "Unknown";
        price = 0.0;
        quantity = 0;
    }
    Product(String name, double price) {
        this.name = name;
        this.price = price;
        this.quantity = 1;
    }
    Product(String name, double price, int quantity) {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }
    double getTotalCost() {
        return price * quantity;
    }
    static double applyDiscount(double total) {
        if (total > 15000)
            return total - total * 0.25;
        else if (total > 10000)
            return total - total * 0.20;
        else if (total > 5000)
            return total - total * 0.15;
        else
            return total;
    }
    void displayProduct() {
        System.out.println(name + " | Price: " + price +
            " | Quantity: " + quantity +
            " | Total: " + (price * quantity));
    }
}

public class Ecommerce {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Product p1 = new Product("Powerbank", 800);
        Product p2 = new Product("Headphones", 150, 2);
        System.out.print("Enter product name: ");
        String name = sc.nextLine();
        System.out.print("Enter product price: ");
        double price = sc.nextDouble();
        System.out.print("Enter quantity: ");
        int quantity = sc.nextInt();
        Product userProduct = new Product(name, price, quantity);
        System.out.println("\nOrder Summary:");
        p1.displayProduct();

        p2.displayProduct();
        userProduct.displayProduct();
        double totalCost = p1.getTotalCost()
            + p2.getTotalCost()
            + userProduct.getTotalCost();
        double finalAmount = Product.applyDiscount(totalCost);
        System.out.println("\nTotal Cost Before Discount: " + totalCost);
        System.out.println("Final Price After Discount: " + finalAmount);
        System.out.println("Thank you for shopping!");
        sc.close();
    }
}
```

Sample Output:

Sample Output:

Enter product name: Laptop

Enter product price: 20000

Enter quantity: 1

Order Summary:

Powerbank | Price: 800.0 | Quantity: 1 | Total: 800.0

Headphones | Price: 150.0 | Quantity: 2 | Total: 300.0

Laptop | Price: 20000.0 | Quantity: 1 | Total: 20000.0

Total Cost Before Discount: 21100.0
Final Price After Discount: 15825.0
Thank you for shopping!

practical 3 (part A).pdf

Java Method Overloading Program - Code and Output

Program Code:

```
import java.lang.Math;
class OverloadingDemo {
    int power(int base, int exp) {
        int result = 1;
        for (int i = 1; i <= exp; i++) {
            result *= base;
        }
        return result;
    }
    double power(double base, int exp) {
        double result = 1;
        for (int i = 1; i <= exp; i++) {
            result *= base;
        }
        return result;
    }
    int absolute(int num) {
        return Math.abs(num);
    }
    double absolute(double num) {
        return Math.abs(num);
    }
}
public class TestOverloadingDemo {
    public static void main(String[] args) {
        OverloadingDemo c = new OverloadingDemo();
        System.out.println("Power (int): " + c.power(2, 3));
        System.out.println("Power (double): " + c.power(2.5, 3));
        System.out.println("Absolute (int): " + c.absolute(-10));
        System.out.println("Absolute (double): " + c.absolute(-15.75));
    }
}
```

Sample Output:

Sample Output:

Power (int): 8

Power (double): 15.625

Absolute (int): 10

Absolute (double): 15.75

practical 4 (part B).pdf

ATM Simulator (Java) - Code & Output

Java Code:

```
import java.util.Scanner;
class ATM {
    private double balance;
    ATM(double initialBalance) {
        balance = initialBalance;
    }
    void checkBalance() {
        System.out.println("Current Balance: ■" + balance);
    }
    void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive.");
        }
        balance += amount;
        System.out.println("■" + amount + " deposited successfully.");
    }
    void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive.");
        }
        if (amount > balance) {
            throw new ArithmeticException("Insufficient funds.");
        }
        balance -= amount;
        System.out.println("■" + amount + " withdrawn successfully.");
    }
}

public class ATMSimulator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ATM atm = new ATM(5000);
        int choice;
        do {
            System.out.println("\n----- ATM MENU -----");
            System.out.println("1. Check Balance");
            System.out.println("2. Deposit Money");
            System.out.println("3. Withdraw Money");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
            try {
                choice = sc.nextInt();
                switch (choice) {
                    case 1:
                        atm.checkBalance();
                        break;
                    case 2:
                        System.out.print("Enter deposit amount: ");
                        double depositAmount = sc.nextDouble();
                        atm.deposit(depositAmount);
                        break;
                    case 3:
                        System.out.print("Enter withdrawal amount: ");
                        double withdrawAmount = sc.nextDouble();
                        atm.withdraw(withdrawAmount);
                        break;
                    case 4:
                        System.out.println("Thank you for using ATM.");
                        sc.close();
                        return;
                    default:
                        throw new IllegalArgumentException("Invalid menu choice.");
                }
            } catch (ArithmeticException e) {
                System.out.println("Error: " + e.getMessage());
            } catch (IllegalArgumentException e) {
                System.out.println("Error: " + e.getMessage());
            } catch (Exception e) {
                System.out.println("Error: Invalid input. Please enter numbers only.");
                sc.nextLine();
            } finally {
                System.out.println("Transaction completed.\n");
            }
        } while (true);
    }
}
```

```
}
Sample Output:
----- ATM MENU -----
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 1
Current Balance: ■5000.0
Transaction completed.
----- ATM MENU -----
Enter your choice: 2
Enter deposit amount: 1000
■1000.0 deposited successfully.
Transaction completed.
----- ATM MENU -----
Enter your choice: 3
Enter withdrawal amount: 2000
■2000.0 withdrawn successfully.
Transaction completed.
----- ATM MENU -----
Enter your choice: 4
Thank you for using ATM.
Transaction completed.
```

practical 5 (part A).pdf

Array Operations in Java (Complete Code + Sample Output)

Java Code:

```
import java.util.Scanner;
class ArrayMenu {
    int arr[];
    int n;
    ArrayMenu(int size) {
        n = size;
        arr = new int[n];
    }
    void inputArray() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
    }
    void display() {
        System.out.println("Array Elements:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
    void findMax() {
        int max = arr[0];
        for (int i = 1; i < n; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }
        System.out.println("Maximum Element: " + max);
    }
    void findMin() {
        int min = arr[0];
        for (int i = 1; i < n; i++) {
            if (arr[i] < min) {
                min = arr[i];
            }
        }
        System.out.println("Minimum Element: " + min);
    }
    void findSum() {
        int sum = 0;
        for (int i = 0; i < n; i++) {
            sum += arr[i];
        }
        System.out.println("Sum of Elements: " + sum);
    }
    void findAverage() {
        int sum = 0;
        for (int i = 0; i < n; i++) {
            sum += arr[i];
        }
        double avg = (double) sum / n;
        System.out.println("Average of Elements: " + avg);
    }
    void searchElement() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter element to search: ");
        int key = sc.nextInt();
        boolean found = false;
        for (int i = 0; i < n; i++) {
            if (arr[i] == key) {
                System.out.println("Element found at position: " + (i + 1));
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Element not found");
        }
    }
}
```

```

public class ArrayOperations {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter array size: ");
        int size = sc.nextInt();
        ArrayMenu obj = new ArrayMenu(size);
        obj.inputArray();
        int choice;
        do {
            System.out.println("\n--- Array Operations Menu ---");
            System.out.println("1. Display Array");
            System.out.println("2. Find Maximum");
            System.out.println("3. Find Minimum");
            System.out.println("4. Find Sum");
            System.out.println("5. Find Average");
            System.out.println("6. Search Element");
            System.out.println("7. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    obj.display();
                    break;
                case 2:
                    obj.findMax();
                    break;
                case 3:
                    obj.findMin();
                    break;
                case 4:
                    obj.findSum();
                    break;
                case 5:
                    obj.findAverage();
                    break;
                case 6:
                    obj.searchElement();
                    break;
                case 7:
                    System.out.println("Exiting Program...");
                    break;
                default:
                    System.out.println("Invalid choice!");
            }
        } while (choice != 7);
    }
}

```

Sample Output:

```

Sample Output:
Enter array size: 5
Enter 5 elements:
10 20 30 40 50
--- Array Operations Menu ---
1. Display Array
2. Find Maximum
3. Find Minimum
4. Find Sum
5. Find Average
6. Search Element
7. Exit
Enter your choice: 1
Array Elements:
10 20 30 40 50
Enter your choice: 2
Maximum Element: 50
Enter your choice: 3
Minimum Element: 10
Enter your choice: 4
Sum of Elements: 150
Enter your choice: 5
Average of Elements: 30.0
Enter your choice: 6
Enter element to search: 30
Element found at position: 3
Enter your choice: 7
Exiting Program...

```

practical 6 (part B).pdf

Stock Monitor System - Java Multithreading
Program

Complete Code:

```
import java.util.Random;
class Stock {
    private double price;
    synchronized void setPrice(double price) {
        this.price = price;
    }
    synchronized double getPrice() {
        return price;
    }
}
class PriceFetcher extends Thread {
    private Stock stock;
    PriceFetcher(Stock stock) {
        this.stock = stock;
    }
    public void run() {
        Random random = new Random();
        for (int i = 1; i <= 5; i++) {
            try {
                double newPrice = 100 + random.nextInt(50);
                stock.setPrice(newPrice);
                System.out.println("Fetched stock price: ■" + newPrice);
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println("Fetcher thread interrupted");
            }
        }
    }
}
class PriceDisplay extends Thread {
    private Stock stock;
    PriceDisplay(Stock stock) {
        this.stock = stock;
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            try {
                System.out.println("Displayed stock price: ■" + stock.getPrice());
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println("Display thread interrupted");
            }
        }
    }
}
public class StockMonitor {
    public static void main(String[] args) {
        Stock stock = new Stock();
        PriceFetcher fetcher = new PriceFetcher(stock);
        PriceDisplay display = new PriceDisplay(stock);

        fetcher.start();
        display.start();
        try {
            fetcher.join();
            display.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted");
        }
        System.out.println("Stock monitoring completed.");
    }
}
```

Sample Output:

Sample Output:

```
Fetched stock price: ■120.0
Displayed stock price: ■120.0
Fetched stock price: ■135.0
Displayed stock price: ■135.0
Fetched stock price: ■110.0
Displayed stock price: ■110.0
Fetched stock price: ■145.0
Displayed stock price: ■145.0
Fetched stock price: ■130.0
```

Displayed stock price: ■130.0
Stock monitoring completed.